

RAGtune

Budget-Aware Feedback-Driven Retrieval

Breaking the performance cap of iterative reranking

Standard RAG is a one-way street

```
Retrieve N docs → Rerank top-K → LLM
```

The only controls you have:

- **N** — how many docs to retrieve
- **K** — how many to rerank

No adaptation. No feedback. Every doc gets the same treatment regardless of what you learn along the way.

If you have budget for 10 reranks, you pick the top-10 by retrieval score. Done.

Why that's a problem

Reranking is expensive: latency, API calls, token budget.

The retriever gives you a ranked list — but retrieval rank $\neq$ relevance rank.

Retrieval rank	Reranker score
1st	0.91 ✓
2nd	0.23 ✗
3rd	0.87 ✓
11th	0.95 ✓ — never seen

A static pipeline with budget for 10 reranks misses the 11th doc entirely.

Feedback-driven retrieval: the concept

What if reranking was **iterative**?

```
Retrieve → Rerank batch 1 → Learn → Rerank batch 2 → ... → Assemble
```

Each scored batch reveals signal:

- Which docs are actually relevant
- Which retrieval sources are predictive
- Where the relevance boundary is

Already better than one-pass for the same total rerank budget — **in theory**.

The performance cap problem

Here's the catch: **if scheduling is naive, you hit a wall.**

Naive scheduling = sort remaining docs by retrieval score, take top-N.

But retrieval score is exactly the signal you're trying to improve on.

You're not using the feedback you just accumulated to decide what to schedule next.

Result: feedback-driven retrieval with naive scheduling $\approx$ one-pass in sorted order.

The loop runs, but the signal goes to waste.

The core insight

The scheduling problem is non-trivial in a feedback loop.

Each reranked batch tells you:

- What a relevant doc looks like (semantically, structurally)
- Which retrieval source is most predictive for this query
- Where the score boundary sits

Using **that signal** — not retrieval rank — to choose the next batch is where the performance cap breaks.

RAGtune's claim: same budget, higher recall
by making scheduling feedback-aware.

RAGtune: The Loop

```
while budget remains:
    priorities = Estimator.value(pool)           # use feedback to score candidates
    batch      = Scheduler.select_batch()        # allocate budget intelligently
    scores     = Reranker.rerank(batch)          # expensive signal
    pool.update_scores(scores)                   # CANDIDATE → RERANKED / DROPPED
```

Two key separations:

- **Estimator** — how to convert feedback into priorities
- **Scheduler** — how to allocate the remaining budget

Independent policies. Swap either without touching the loop.

CandidatePool: shared state across iterations

Source: `src/ragtune/core/pool.py`

```
CANDIDATE → IN_FLIGHT → RERANKED  
                               ↘  
                               DROPPED
```

Every doc passes through exactly once.

The pool is where signal accumulates — reranker scores, estimator priorities, source provenance. The Estimator reads the RERANKED docs to inform priorities on the CANDIDATE docs.

Estimator: converting feedback into priorities

Question: "Given what we've reranked so far, which remaining docs are worth reranking next?"

Source: `src/ragtune/components/estimators.py`

Estimator	How it uses feedback
Baseline	Doesn't — raw retrieval score only
Utility	Boosts docs sharing metadata with high-scoring winners
Similarity	Cosine similarity to reranked winners via embeddings
ReformIR	Regression: source scores → reranker score; re-weights every iteration

ReformIR: after ≥ 3 docs scored, learns which retrieval source predicts relevance. Priorities shift on every iteration.

Scheduler: budget allocation policy

Question: "Given priorities and remaining budget, what do we run next — and with what tool?"

Source: `src/ragtune/components/schedulers.py`

ActiveLearningScheduler

Top-N by priority. Escalates to LLM when top-2 candidates are within 5% priority — uses a more expensive signal to break the tie.

GracefulDegradationScheduler

Spend LLM calls first (highest fidelity where it matters most), then cross-encoder, then stop.

The seam between Estimator and Scheduler is also where **exploration vs exploitation** logic lives — currently greedy, but the interface is ready.

Budget enforcement

Source: `src/ragtune/core/budget.py`

Four independent constraints — any combination:

Constraint	What it limits
<code>rerank_docs</code>	Total documents through reranker
<code>rerank_calls</code>	API calls to reranker
<code>tokens</code>	Estimated token consumption
<code>latency_ms</code>	Wall-clock hard deadline

Every operation calls `try_consume()` before executing.

Graceful degradation: budget exhausted = return best results found so far. Never crashes, never overspends.

Observability: ControllerTrace

Source: `src/ragtune/core/types.py`

Flat event log — every decision emits `(component, action, details, timestamp)`:

```
{"component": "controller", "action": "pool_init",  
  "details": {"count": 32, "metrics": {"rewrite_utility_ratio": 0.56}}}  
  
{"component": "estimator", "action": "reformir_weights_updated",  
  "details": {"weights": {"original": 0.42, "rewrite_0": 0.71}, "n_reranked": 5}}  
  
{"component": "controller", "action": "rerank_batch",  
  "details": {"strategy": "llm", "doc_ids": [...], "dropped_ids": null}}  
  
{"component": "controller", "action": "feedback_stop",  
  "details": {"reason": "weights_converged: delta=0.003 < threshold=0.01"}}
```

Makes the loop inspectable: see exactly how priorities shifted and why.

Configuration & CLI

Single `ragtune_config.yaml` — declarative pipeline definition:

```
components:
  estimator: { type: "reformir" }
  scheduler: { type: "graceful-degradation", params: { llm_limit: 10 } }
  reranker: { type: "cross-encoder" }
budget:
  rerank_docs: 50
  latency_ms: 2000
```

Full lifecycle:

```
ragtune init --wizard    # interactive config generation
ragtune index config.yaml
ragtune run config.yaml -q "my query"
ragtune visualize config.yaml --edit    # interactive pipeline editor
```

Demo 1: The basic loop

Script: `examples/quickstart.py`

- In-memory corpus, no external dependencies, runs in ~2s
- Shows: budget enforcement, trace events, graceful degradation

```
python examples/quickstart.py
```

Watch for:

- The loop iterating over batches until `rerank_docs` budget exhausted
- The trace showing each `rerank_batch` event
- Strict budget (`tokens=0`) returning 0 documents — not a crash

[LIVE DEMO 1]

```
python examples/quickstart.py
```

Demo 2: Feedback changes what gets scheduled

Script: `examples/demo_active_learning.py`

- 3 docs: `doc_1` (section A, score 0.50), `doc_2` (section A, score 0.40), `doc_3` (section B, score 0.45)
- Budget: 3 rerank docs, batch size 1

Without feedback (naive): schedule order = `doc_1, doc_3, doc_2` (sorted by retrieval score)

Demo 2: What the feedback loop does

With UtilityEstimator:

1. `doc_1` reranked first (highest retrieval score) → confirmed relevant
2. Estimator sees `doc_1` was a winner → boosts all docs in section A
3. `doc_2` priority jumps above `doc_3` despite lower initial score
4. Actual schedule: `doc_1 → doc_2 → doc_3` ✓

`doc_2` leapfrogged `doc_3` — the feedback signal changed what got scheduled next.

```
python examples/demo_active_learning.py
```

[LIVE DEMO 2]

```
python examples/demo_active_learning.py
```

Results: does it work?

Experiment setup: 3 datasets (NFCorpus, SciFact, TREC-COVID), MonoT5 reranker, ablation across budget / estimator / feedback strategy.

Key finding: convergence feedback beats brute-force budget (TREC-COVID)

Config	NDCG@5	Rerank docs	Latency
BM25 only	0.591	0	36ms
MonoT5, 15 docs	0.745	15	1486ms
MonoT5, 30 docs	0.699	30	3067ms
Convergence feedback	0.774	10	992ms

Convergence feedback: **+4% NDCG over the 30-doc run, using 1/3 the docs, in 1/3 the time.**

More docs $\neq$ better results. Smarter scheduling does.

Results: the Pareto picture

Across all 3 datasets, the pattern holds:

Config	Avg NDCG@5	Avg rerank docs	Avg latency
BM25 only	0.656	0	17ms
MonoT5 tight (5 docs)	0.720	5	574ms
MonoT5 medium (15 docs)	0.749	15	1563ms
MonoT5 loose (30 docs)	0.748	30	3117ms
Convergence feedback	0.760	10	1008ms

- Tight budget (5 docs) closes most of the gap vs BM25 baseline
- Convergence feedback matches or beats loose (30 docs) at medium (10 docs) cost
- **The ceiling isn't the budget — it's how you use it**

Summary

Standard RAG: one-way pipeline, budget = how many docs to rerank

Feedback-driven RAG: iterative loop — but naive scheduling wastes the signal

RAGtune: same budget, higher performance cap by making scheduling feedback-aware

Component	Role
Estimator	Converts reranked signal into priorities for remaining candidates
Scheduler	Allocates budget given priorities (which docs, which tool)
CostTracker	Enforces hard constraints (docs, tokens, calls, latency)
ControllerTrace	Full audit log — every decision is inspectable

The next step: true exploration vs exploitation in the scheduler — and uncertainty-aware estimators (U-SIR).

Thank you

Code: `github.com/avishekanand/sir`

Key files:

- Loop: `src/ragtune/core/controller.py`
- Estimators: `src/ragtune/components/estimators.py`
- Schedulers: `src/ragtune/components/schedulers.py`
- Demos: `examples/quickstart.py`, `examples/demo_active_learning.py`

Render these slides:

```
marp docs/presentation.md --pdf  
# or: marp docs/presentation.md --html
```